Lab 05 — Hashing, Salting, and Digital Signatures

> **SY0-701 Objective 1.4** — Explain the importance of using appropriate cryptographic solutions.
>
> **Domain 1** — General Security Concepts (12%) · **Time:** ~45 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Prove data integrity with hashes, protect passwords with salting and key stretching, add authenticity with an HMAC, and achieve non-repudiation with a digital signature that fails the moment the document is altered.

You'll build: A SHA-256 checksum manifest that detects tampering, salted SHA-512 and yescrypt password hashes, a PBKDF2 key-stretching timing comparison, an HMAC-protected invoice, and an RSA-3072 signed purchase order with a verified signature.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 02 (integrity basics) is helpful; tools installed in step 1.

Background

- A **hash** is a one-way, fixed-length fingerprint. It gives **integrity** only: anyone can recompute it, so it says nothing about *who* produced the data.
- **Salting** adds a unique random value to each password before hashing, which defeats rainbow tables and hides users who chose the same password.
- **Key stretching** (PBKDF2, bcrypt, scrypt, Argon2, yescrypt) repeats the work thousands of times so each guess is slow for an attacker.
- An **HMAC** mixes a shared secret key into the hash, giving integrity plus authenticity between parties who share the key.
- A **digital signature** is a hash signed with the sender's **private key** and verified with their **public key**: integrity, authenticity and **non-repudiation**.

Steps

1. Install the tools

OpenSSL 3 provides hashing, HMAC, PBKDF2 and signatures; coreutils provides `sha256sum`.

```
apt-get update -qq && apt-get install -y openssl coreutils
openssl version
mkdir -p /root/lab05 && cd /root/lab05
```

2. Hash data and observe fixed length and the avalanche effect

Two messages differ by one character. Their digests are the same length but completely different. MD5 is shown only for comparison: it is broken for security use.

```
cd /root/lab05
echo -n 'Pay Bob 100 SGD' | sha256sum
echo -n 'Pay Bob 900 SGD' | sha256sum
echo -n 'Pay Bob 100 SGD' | md5sum
echo -n 'Pay Bob 100 SGD' | sha256sum | cut -d' ' -f1 | tr -d '\n' | wc -c
```

SHA-256 always gives 64 hex characters (256 bits), whatever the input size.

3. Verify file integrity with a checksum manifest

Software vendors publish a `SHA256SUMS` file so you can confirm downloads were not corrupted or modified.

```
cd /root/lab05
echo "release notes v1.0" > notes.txt
echo "installer payload v1.0" > installer.bin
sha256sum notes.txt installer.bin > SHA256SUMS
sha256sum -c SHA256SUMS
echo "malicious bytes" >> installer.bin
sha256sum -c SHA256SUMS || echo "INTEGRITY FAILED - do not install"
```

4. See why unsalted password hashes are weak

Two users chose the same password. Without a salt their hashes are identical, so cracking one cracks both, and a precomputed rainbow table works instantly.

```
cd /root/lab05
printf 'carol:%s\ndave:%s\n' "$(echo -n 'Summer2026!' | sha256sum | cut -d' ' -f1)" "$(echo -n
'Summer2026!' | sha256sum | cut -d' ' -f1)" > unsalted.txt
cat unsalted.txt
cut -d: -f2 unsalted.txt | sort | uniq -d | sed 's/^/DUPLICATE HASH: /'
```

5. Salt the hash the way /etc/shadow does

Add a random salt and the same password produces different hashes. Real Linux accounts store the algorithm, salt and hash together in `/etc/shadow`.

```
openssl passwd -6 -salt "$(openssl rand -hex 8)" 'Summer2026!'
openssl passwd -6 -salt "$(openssl rand -hex 8)" 'Summer2026!'
for u in carol dave; do id "$u" >/dev/null 2>&1 || useradd -m "$u"; echo "$u:Summer2026!" | chpasswd; done
getent shadow carol dave | cut -d: -f1,2
```

```
carol:$y$j9T$<salt-A>$<hash-A>
dave:$y$j9T$<salt-B>$<hash-B>
```

`$6$` is SHA-512 crypt and `$y$` is yescrypt, the Ubuntu default. Different salts mean different hashes for the same password.

6. Slow attackers down with key stretching

PBKDF2 repeats an HMAC many times. Compare 1,000 iterations with 600,000: the legitimate user waits a fraction of a second once, while an attacker pays that cost for every single guess.

```
time openssl kdf -keylen 32 -kdfopt digest:SHA256 -kdfopt 'pass:Summer2026!' -kdfopt salt:Lab05UniqueSalt01
-kdfopt iter:1000 PBKDF2
time openssl kdf -keylen 32 -kdfopt digest:SHA256 -kdfopt 'pass:Summer2026!' -kdfopt salt:Lab05UniqueSalt01
-kdfopt iter:600000 PBKDF2
```

Note the `real` time of each run. The derived keys differ because the iteration count is part of the calculation.

7. Add authenticity with an HMAC

A plain hash can be recomputed by an attacker who alters the file. An HMAC needs the shared secret key, so only key holders can produce a valid tag.

```
cd /root/lab05
echo "Invoice 4471: amount 2500 SGD to account 123-456" > invoice.txt
openssl dgst -sha256 -hmac 'SharedLabKey-2026' invoice.txt | tee invoice.hmac
sed -i 's/123-456/999-999/' invoice.txt
openssl dgst -sha256 -hmac 'SharedLabKey-2026' invoice.txt
```

The tag changes after tampering, and an attacker without `SharedLabKey-2026` cannot forge a matching one. Both parties hold the key, though, so an HMAC does not give non-repudiation.

8. Sign a document with a private key

Generate an RSA-3072 key pair, keep the private key secret, publish the public key, and sign the SHA-256 digest of a purchase order.

```
cd /root/lab05
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out signer.key 2>/dev/null
chmod 600 signer.key
openssl pkey -in signer.key -pubout -out signer.pub
echo "PO-7781: 20 laptops, total 36000 SGD, approved by Alice" > order.txt
openssl dgst -sha256 -sign signer.key -out order.sig order.txt
openssl dgst -sha256 -verify signer.pub -signature order.sig order.txt
```

```
Verified OK
```

9. Tamper with the document and watch verification fail

Change one value. The signature no longer matches, proving integrity; and because only Alice holds `signer.key`, a valid signature proves she signed it (non-repudiation).

```
cd /root/lab05
sed -i 's/20 laptops/200 laptops/' order.txt
openssl dgst -sha256 -verify signer.pub -signature order.sig order.txt || echo "SIGNATURE INVALID -
document altered after signing"
```

```
Verification failure
SIGNATURE INVALID - document altered after signing
```

Test it

```
cd /root/lab05
sha256sum -c SHA256SUMS 2>/dev/null | grep -c FAILED
getent shadow carol dave | cut -d: -f2 | sort -u | wc -l
echo "PO-7781: 20 laptops, total 36000 SGD, approved by Alice" > order.txt
openssl dgst -sha256 -verify signer.pub -signature order.sig order.txt
```

Expected: `1` (the tampered installer is caught), `2` (two different salted hashes for one password), and `Verified OK` once the original order text is restored.

Troubleshooting

Symptom	Fix
`openssl kdf: Unknown command` or `Invalid command 'kdf'`	`kdf` needs OpenSSL 3.0 or later (Ubuntu 22.04+). Check `openssl version`; on older images use `python3 -c "import hashlib;print(hashlib.pbkdf2_hmac('sha256',b'Summer2026!',b'Lab05UniqueSalt01',600000).hex())"`.
`Verification failure` on the untouched file	The signature and file must be byte-identical to what was signed, including the trailing newline. Recreate `order.txt` with the exact `echo` line and sign again.
`getent shadow` shows `!` for a user	The password was not set; rerun the `chpasswd` loop in step 5.

Challenge

Use an **elliptic-curve** key instead of RSA: generate a P-256 key with `openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256`, sign and verify the order, then compare the size of the two signature files. Explain which property of

ECC makes it attractive for mobile and IoT devices.

Reflection

A hash, an HMAC and a digital signature can all detect that a file changed. Which of the three can also prove *who* created the file to a third party, and why can the other two not?

Exam tips

- Hashing is **one-way** and gives integrity; encryption is **reversible** and gives confidentiality. Hashing is not encryption.
- **Salt** defeats rainbow tables; **key stretching** slows brute force. Fast hashes (MD5, plain SHA-256) are the wrong choice for passwords.
- You **sign with your private key**; others **verify with your public key**. Signatures give non-repudiation; HMACs do not.
- MD5 and SHA-1 have known collisions; choose SHA-256 or stronger for integrity and signatures.

References

- NIST FIPS 180-4 Secure Hash Standard — https://csrc.nist.gov/pubs/fips/180-4/upd1/final
- NIST SP 800-132 Password-Based Key Derivation — https://csrc.nist.gov/pubs/sp/800/132/final
- OWASP Password Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OpenSSL `kdf`, `dgst` and `genpkey` manuals — https://docs.openssl.org/3.0/man1/
